

Day 4 — Selecting Data

Class notes - tips.csv + Titanic-Dataset.csv

These are the notes for what we did in class today. Read them with the class notebook open.

1. Picking columns

You write	You get
<code>df['tip']</code>	a Series - one column
<code>df[['tip']]</code>	a DataFrame - a table with one column
<code>df[['day','tip']]</code>	a table with two columns
<code>df['new'] = ...</code>	creates a new column

One bracket gives you a column. Two brackets give you a table. The inner brackets are a *list of names* - that is the whole trick.

```
df['total_paid'] = df['total_bill'] + df['tip']
df['country'] = 'India' # one value fills the whole column
```

Assigning a calculation runs it row by row automatically. There is no loop anywhere in Pandas code like this.

2. iloc - picking by position

`iloc` means *integer location*. It understands numbers and nothing else.

```
df.iloc[4] # one row
df.iloc[6:10:2] # a slice, with a step
df.iloc[[6,7,8,10]] # a list of positions
df.iloc[2, [1,3,5]] # row 2, columns at positions 1, 3, 5
df.iloc[-1] # the last row
```

3. loc - picking by label

```
df.loc[100] # the row labelled 100
df.loc[100, 'tip'] # one single value
df.loc[5:8, ['day','tip']] # rows by label, columns by name
df.loc[5:10, 'total_bill':'time'] # slice the COLUMNS by name too
```

That last line is something `iloc` cannot do. With `iloc` you would have to count the column positions yourself.

The difference that catches everyone

	<code>loc</code>	<code>iloc</code>
Picks by	label (name)	position (number)
Slice end	included	not included
Columns as	names	numbers
Takes a condition	yes	no

`df.loc[10:20]` gives 11 rows. `df.iloc[10:20]` gives 10. Remember it as: `loc` = label = includes.

4. Boolean indexing - picking by a condition

A condition on its own does not filter. It produces a True/False value for **every row**.

```
df['tip'] > 6 # 244 True/False values
df[df['tip'] > 6] # keep only the True ones
(df['tip'] > 6).sum() # count them - True counts as 1
df[(df['size'] > 2) & (df['tip'] > 4)] # & both must be true
df[(df['time'] == 'Dinner') | (df['size'] > 4)] # | either one
df[~(df['day'] == 'Sun')] # ~ not
```

Every condition needs its own brackets. Python runs `&` before `>`, so leaving them out breaks the line with a confusing error.

5. loc with a condition - the most used line in Pandas

Rows chosen by a condition, columns chosen by name, one line.

```
df.loc[df['tip'] > 6, ['day', 'tip']]
df.loc[df['day'] == 'Sun', 'total_bill'].mean()
```

6. Counting with value_counts()

```
df['day'].value_counts()          # count each value
df[df['time'] == 'Dinner']['day'].value_counts()  # filter first, then count
```

One line instead of one condition per value. In our data the Dinner-by-day count came out **Thur = 1** - the restaurant served exactly one dinner on a Thursday all week. That is real, not a bug.

7. Changing values

The same line as selecting, with an = at the end. Reading and writing use identical syntax.

```
df['free_dessert'] = 'No'          # 1. default for everyone
df.loc[df['size'] >= 6, 'free_dessert'] = 'Yes'  # 2. overwrite where true
df['free_dessert'].value_counts()  # 3. check
```

```
df.loc[5, 'tip'] = 5.00          # fix one single cell
```

Spelling is not checked. 'big' and 'Big' are two different labels and Pandas will never warn you. Pick one and stay with it.

8. The Titanic, and what broke

Column names in this file start with capitals: Survived, Pclass, Sex, Age, SibSp, Parch, Fare, Cabin, Embarked. There is no 'travelling alone' column - you build it with (t['SibSp'] == 0) & (t['Parch'] == 0).

Then the ending. **Age has 177 blanks**. 150 passengers were over 40. 564 were 40 or under. That adds to **714**, not 891.

177 people were not counted as old, not counted as young, not counted at all - and nothing on screen said so. No error, no warning. The answer looked clean and was wrong.

```
t['Age'].isnull().sum()          # 177
t.isnull().sum()                 # Age 177, Cabin 687, Embarked 2
t['Age'].mean()                  # 29.70 - divided by 714, not 891
```

Cabin is worse: **687 blanks out of 891**. More than three quarters of the column does not exist.

Before the next class

- Re-run the class notebook from top to bottom and fill in every cell you left blank.
- Run `t.info()` and `t.isnull().sum()` yourself and look at what comes back.
- Read what NaN is, and why `np.nan == np.nan` is **False**.
- Finish the assignment - 10 questions, nothing from the Bonus section needed.

Tomorrow - Day 5, Data Cleaning: dropping rows vs filling blanks, `dropna` and `fillna`, which of mean / median / mode is defensible for Age, when a column like Cabin is beyond saving, duplicates, sorting, and a dataset built out of your own answers.